



AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH(AIUB)

CSC 2210 — Object Oriented Programming 2

Summer 2025-2026

SECTION: D

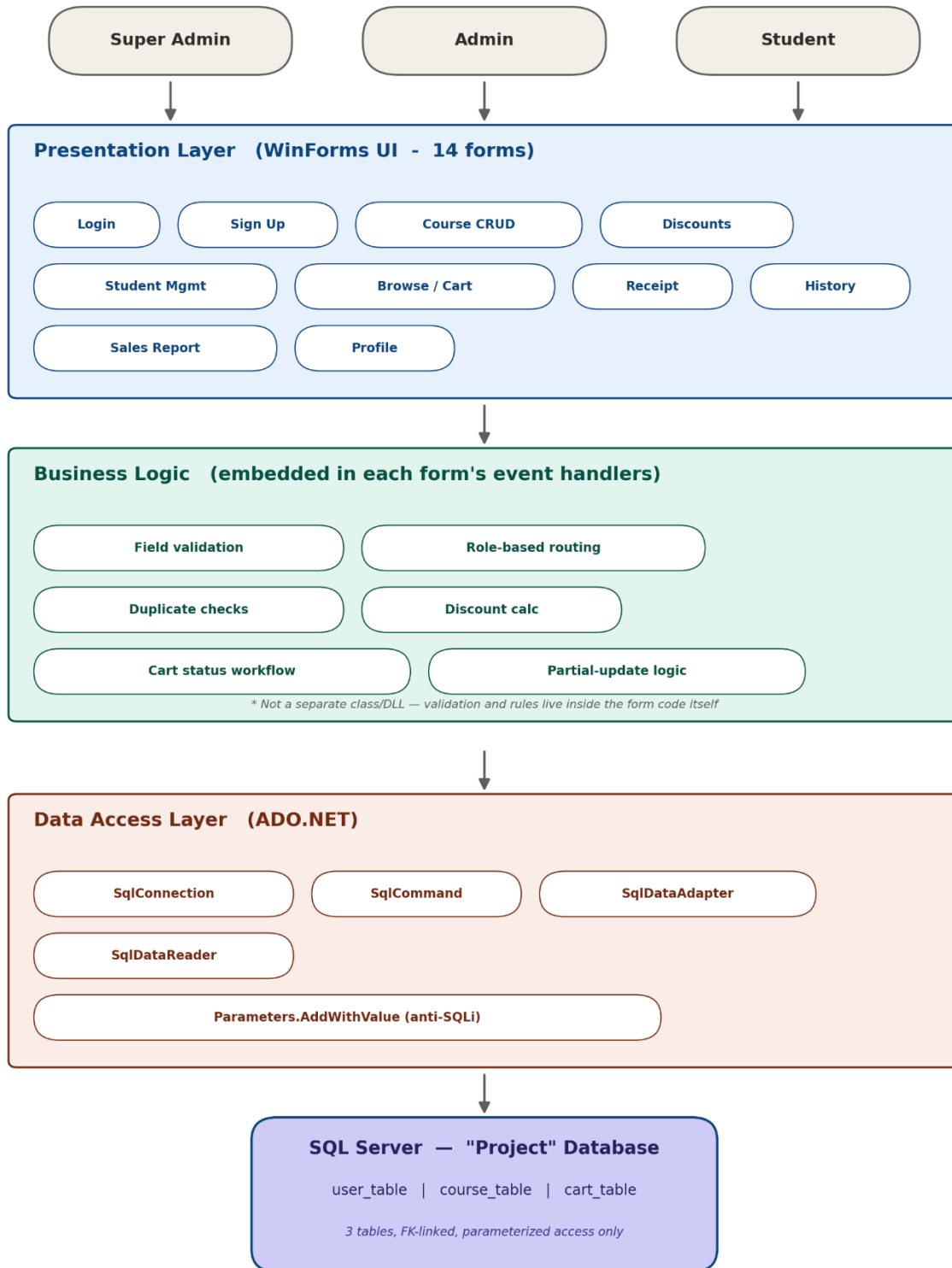
Project Name: Course Management System

Supervised by: DR. MD IFTEKHARUL MOBIN

Name	ID
1. Udipto Kumar Bhowmik	24-57618-2
2. MUBASHSHIRA JANNAT MIMI	23-51475-1
3. MEHJABIN FAIZA	23-51474-1
4. SIAM Farajee	23-51173-1

The System Model:

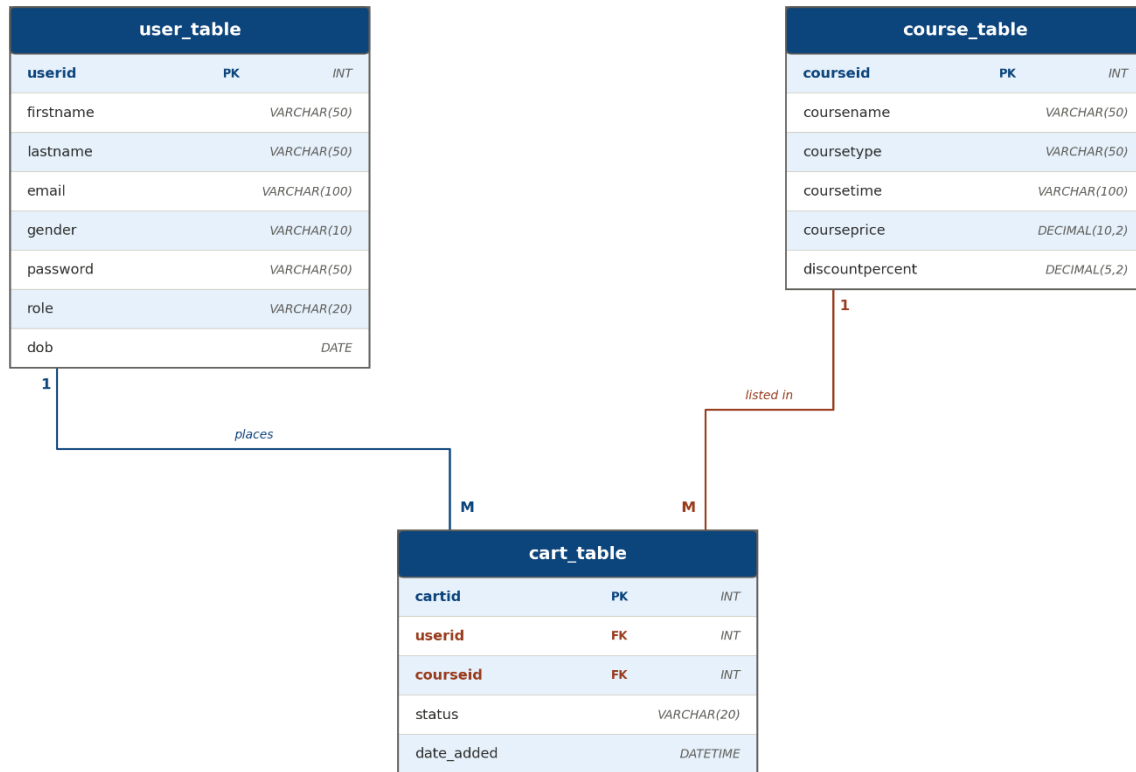
Course Management System — System Architecture



Connection: Data Source=localhost; Initial Catalog=Project; Integrated Security=True

Database Schema

Course Management System — Database Schema (ERD)



cart_table.status flows: Pending → Approved / Rejected → Purchased | cartid is IDENTITY(1,1) auto-increment

Role 1 — SUPER ADMIN

The **Super Admin** is the platform owner and has the highest level of authority. The Super Admin manages and monitors the entire Course Management System.

The Super Admin can:

- Approve or reject new Course Provider registrations.
- Suspend or delete Course Provider accounts when necessary.
- View and search all registered users.
- Manage course categories.
- View platform-wide sales and revenue information.
- Monitor the commission earned by the platform.

- View low-rated courses or course providers.
- Moderate and remove inappropriate or fake reviews.
- Manage their own profile and password.

Role 2 — ADMIN / COURSE PROVIDER

The **Admin** represents a teacher, tutor, coaching center, or educational organization that provides courses through the platform. Each Course Provider can manage only their own courses and related information.

The Course Provider can:

- Register and manage their course-provider profile.
- Add new courses.
- View their own courses.
- Update course information.
- Delete their own courses.
- Manage course price, schedule, and available seats.
- View student enrollments and sales.
- View their earnings.
- Create special discount offers and packages.
- Update their profile and password.

Data isolation is an important requirement. A Course Provider must only be able to access their own courses, enrollments, reviews, and earnings. One provider must not be able to view or modify another provider's private information.

Role 3 — CUSTOMER / STUDENT

The **Customer** represents a student who uses the platform to find and enroll in courses. Students can browse courses offered by different Course Providers through the system.

The Student can:

- Sign up and sign in.
- Browse available courses.
- Search courses by name or keyword.
- Filter courses by category, price, and availability.

- View detailed course information.
- Read ratings and reviews.
- Add courses to the cart.
- Update or remove courses from the cart.
- Complete checkout and enrollment.
- Receive an invoice after completing a purchase.
- View previous orders or enrollments.
- View special offers and discounted courses.
- Update their profile and password.

System Flow

The overall system works as a platform between **Students** and **Course Providers**.

Students → **Course Management System** ← **Course Providers**

The **Super Admin** operates and moderates the platform.

Students browse and purchase/enroll in courses, while **Course Providers** create and manage courses. When a student completes a paid course enrollment, the system records the transaction and calculates the platform's commission. The remaining amount represents the Course Provider's earnings.

The system therefore provides a centralized platform where students can discover suitable courses and Course Providers can manage their courses, students, sales, and earnings, while the Super Admin maintains control over the entire platform.

Chapter 1 — Introduction & Case Study

1.1 Introduction

The Course Management System is a desktop-based application designed to provide a centralized platform where students can discover, request, purchase, and manage courses offered by different teachers or course providers. The system will be developed using C# Windows Forms and a relational database.

The main purpose of the system is to make course management easier for students and course providers. Students can browse available courses, search for courses, view course information, add courses to a cart, complete enrollment or purchase, and provide ratings and reviews. Course providers can manage their own courses, monitor course availability, view sales and earnings,

and create special offers. A Super Admin will control the overall platform, approve course providers, manage categories, monitor platform activity, and moderate reviews.

The system will contain three main user roles: Super Admin, Admin/Course Provider, and Customer/Student. The three-role structure follows the project requirements, where a single user system determines which dashboard a user can access after login.

1.2 Case Study

Many students currently find courses through different sources such as social media pages, individual teachers, coaching centers, and personal communication. This can make it difficult to compare courses, find suitable schedules, check availability, and keep records of enrolled courses. Course providers also face difficulties managing course information, student requests, enrollments, payments, and reviews using separate systems or manual processes. The Course Management System aims to solve these problems by providing one centralized platform for students and course providers.

The system will contain three main actors. The Super Admin will operate and moderate the platform. The Admin, or Course Provider, will represent a teacher, coaching center, or educational organization that provides courses. The Customer, or Student, will use the platform to search for and enroll in courses. Each user will have an account containing information such as user ID, name, email, password, phone number, address, user type, and account status.

Courses will be important entities in the system. Each course will have a unique course ID, course name, category, course type, description, price, available seats, minimum availability level, course schedule, and course provider. A course provider can offer multiple courses, while each course belongs to one provider. Students can browse courses from different providers and use search and filtering options to find suitable courses.

Students will be able to add courses to a cart before checkout. An order or enrollment will contain information about the student, order date, total amount, payment method, and status. Individual course items will be stored separately so that one order can contain multiple courses. Students can also submit ratings and written reviews after taking or purchasing a course.

The platform may earn a commission from course transactions made through the system. When a student purchases or enrolls in a paid course, the platform records the transaction and calculates the platform commission while the remaining amount represents the provider's earnings. Special offers can also be created for selected courses for a specific percentage discount and date range.

The system will therefore connect students and course providers through a single platform while allowing the Super Admin to manage and monitor the overall system.

Chapter 2 — Functional Requirements & User Stories

2.1 Super Admin Requirements

1. Super Admin can log in to the system.
2. Super Admin can approve or reject new course provider registrations.
3. Super Admin can suspend or delete course provider accounts.
4. Super Admin can view all registered users.
5. Super Admin can search and filter users.
6. Super Admin can view platform-wide sales and commission information.
7. Super Admin can view low-rated courses or course providers.
8. Super Admin can manage course categories.
9. Super Admin can moderate and remove inappropriate reviews.
10. Super Admin can update their own profile and password.
11. Super Admin can log out of the system.

2.2 Admin / Course Provider Requirements

1. Course Provider can register an account.
2. Course Provider can manage their profile.
3. Course Provider can add course information.
4. Course Provider can view their courses using a DataGridView.
5. Course Provider can update course information.
6. Course Provider can delete their own courses.
7. Course Provider can manage available seats/course availability.
8. Course Provider can view enrollment and sales information.
9. Course Provider can view earnings.
10. Course Provider can create special discount offers.
11. Course Provider can update their password and profile.
12. Course Provider can log out.

2.3 Customer / Student Requirements

1. Student can register an account.
2. Student can log in.

3. Student can browse available courses.
4. Student can search courses by name or keyword.
5. Student can filter courses by category.
6. Student can filter courses by price range.
7. Student can filter courses by availability.
8. Student can view course details.
9. Student can read course reviews.
10. Student can add courses to the cart.
11. Student can update cart quantity or remove courses.
12. Student can complete checkout.
13. Student can receive an invoice.
14. Student can view order/enrollment history.
15. Student can view special offers.
16. Student can update their profile and password.
17. Student can log out.

2.4 User Stories

Super Admin

User Story 1 — Approve Course Provider

As a Super Admin, I can approve a new course provider, so that only authorized providers can offer courses on the platform.

The Super Admin opens the Manage Course Providers form and views pending registrations in a DataGridView. The administrator can select a provider and click Approve or Reject. The system updates the provider's account status in the database.

User Story 2 — Manage Categories

As a Super Admin, I can add, update, and delete course categories, so that courses can be organized properly.

The Super Admin enters a category name and uses the Add, Update, or Delete buttons. The category name cannot be empty, and duplicate category names should not be accepted. Changes are stored in the Categories table.

User Story 3 — View Platform Sales

As a Super Admin, I can view platform sales and commission, so that I can monitor the financial performance of the platform.

The system displays total sales, number of orders, and commission earned. The information is calculated from completed transactions using SQL aggregate functions and joins.

Admin / Course Provider

User Story 4 — Add Course

As a Course Provider, I can add a new course, so that students can find and enroll in my course.

The provider enters the course name, category, course type, price, available seats, schedule, and description. Required fields cannot be empty and price and availability must contain valid positive values. When the provider clicks Save, the course is inserted with the logged-in provider's ID.

User Story 5 — Update Course

As a Course Provider, I can update my own course information, so that students always see current course details.

The provider selects a course from the DataGridView and clicks Update. The selected information appears in the input controls and can be edited. The database is updated after successful validation.

User Story 6 — View Earnings

As a Course Provider, I can view my sales and earnings, so that I can understand my course performance.

The system displays completed enrollments, course names, dates, quantities, prices, and total earnings. The results are filtered using the logged-in provider's ID so that one provider cannot access another provider's financial information.

Customer / Student

User Story 7 — Search Courses

As a Student, I can search for courses by keyword, so that I can quickly find a course I am interested in.

The student enters a course name or keyword and clicks Search. The system retrieves matching courses from the database and displays them in the course browsing area.

User Story 8 — Filter Courses

As a Student, I can filter courses by category, price, and availability, so that I can find a suitable course more easily.

The student selects values from ComboBox controls and applies the filters. The system returns only courses matching the selected conditions.

User Story 9 — Add Course to Cart

As a Student, I can add a course to my cart, so that I can purchase or enroll in it later.

The student selects a course and clicks Add to Cart. The system checks whether the course is available and stores the selected course in the student's active cart.

User Story 10 — Checkout

As a Student, I can complete checkout, so that I can confirm my course enrollment.

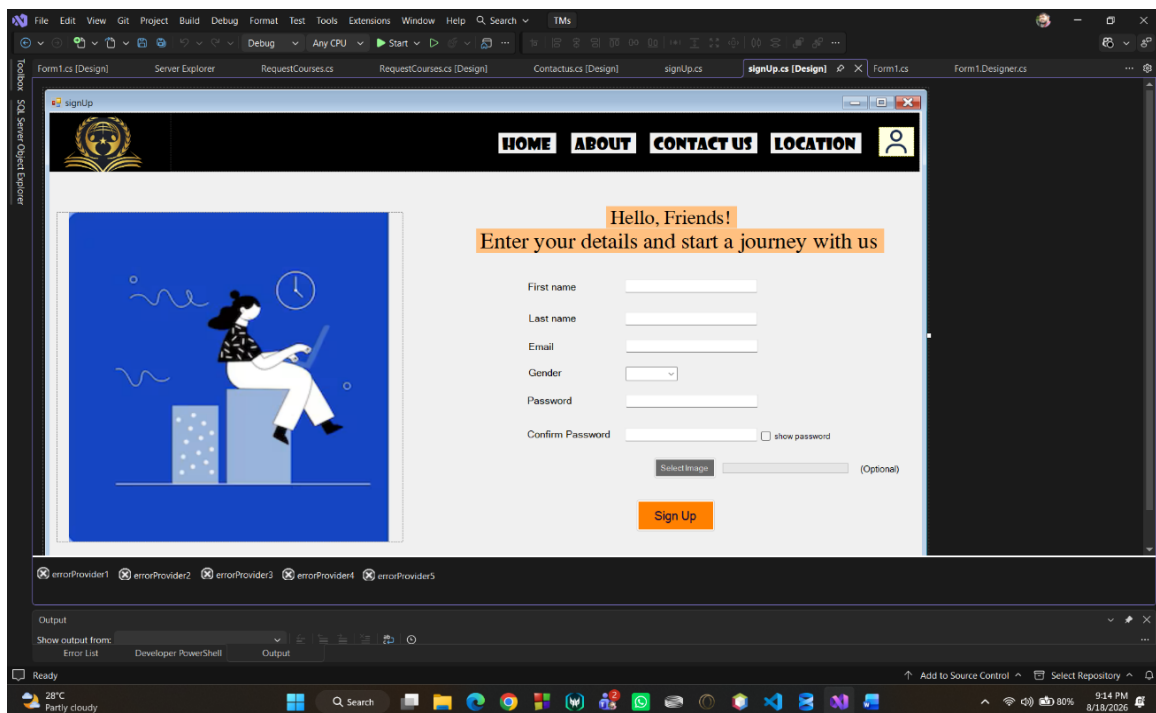
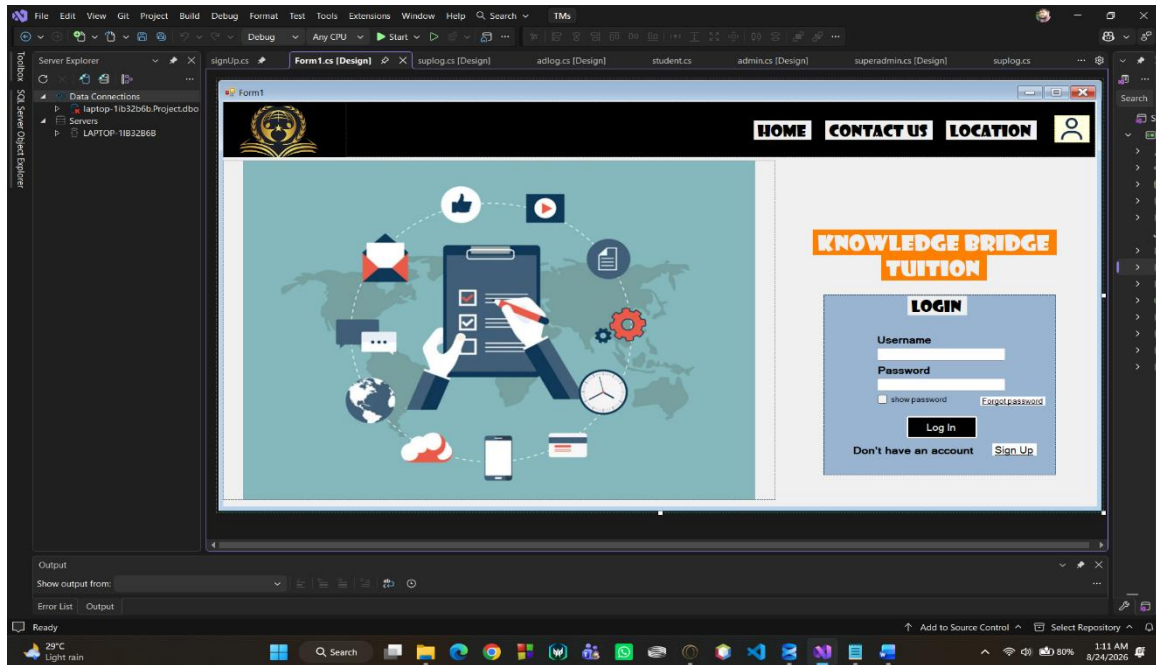
The student reviews the cart, selects a payment method, and confirms the order. The system creates an order and corresponding order items, calculates the total amount, and generates an invoice.

User Story 11 — Review a Course

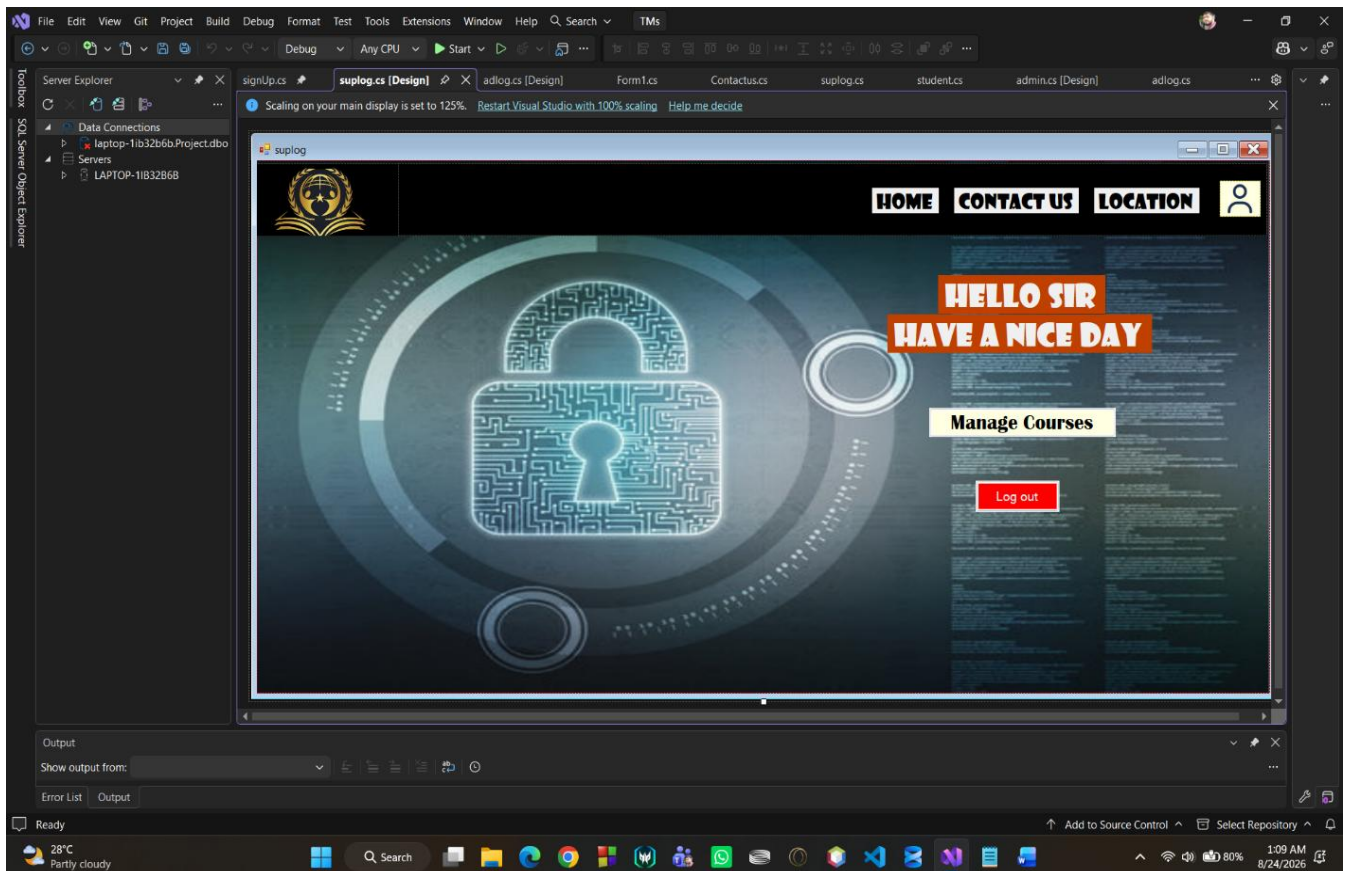
As a Student, I can give a rating and written review, so that I can share my experience with other students.

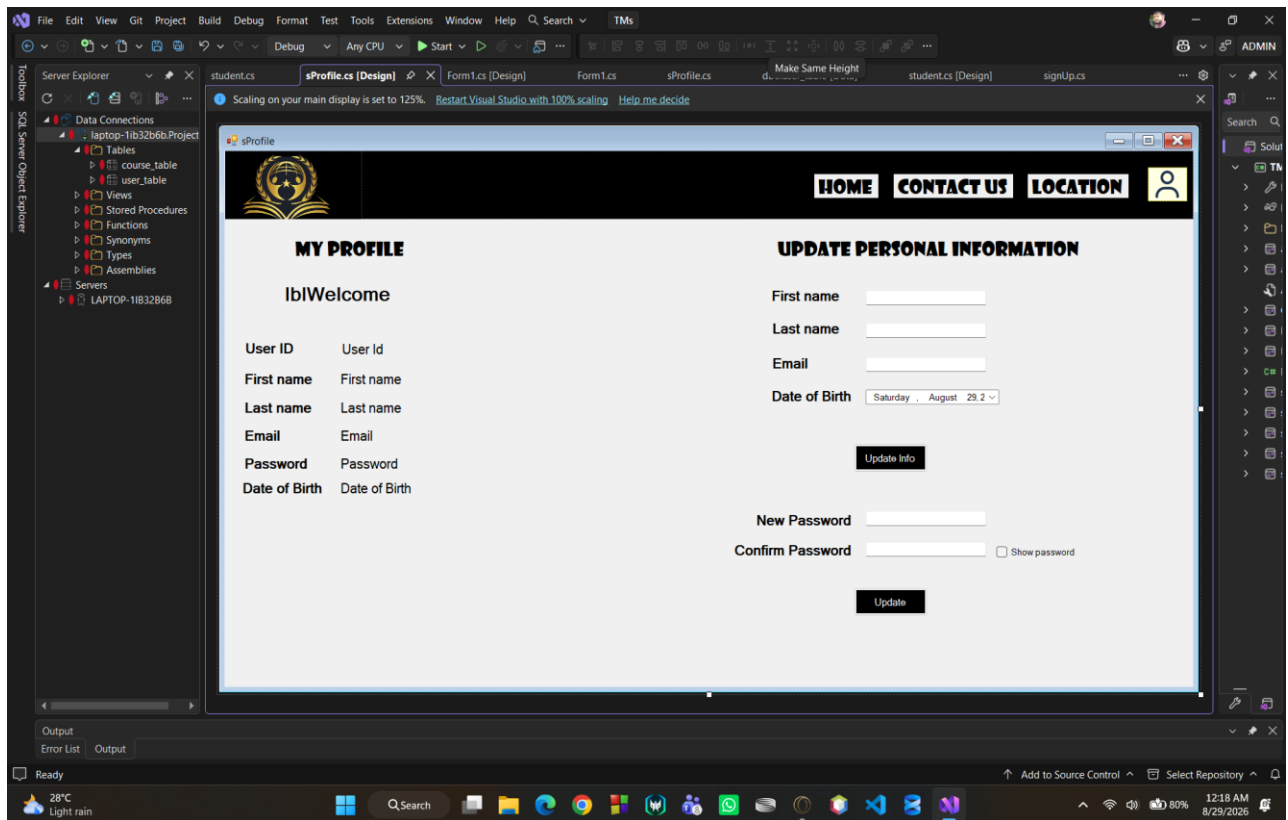
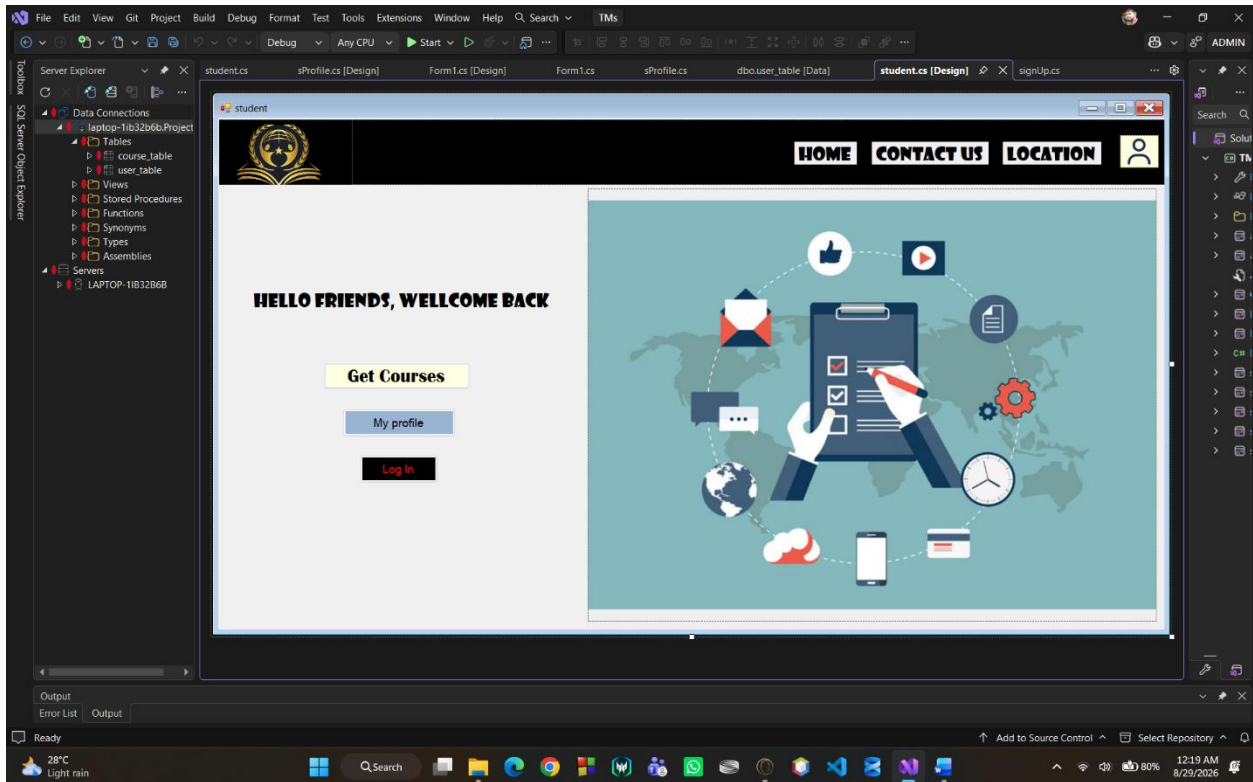
The student selects a rating from 1 to 5 and enters a review. The rating must be within the valid range. The review is stored in the database and can later be viewed by other students and course provider

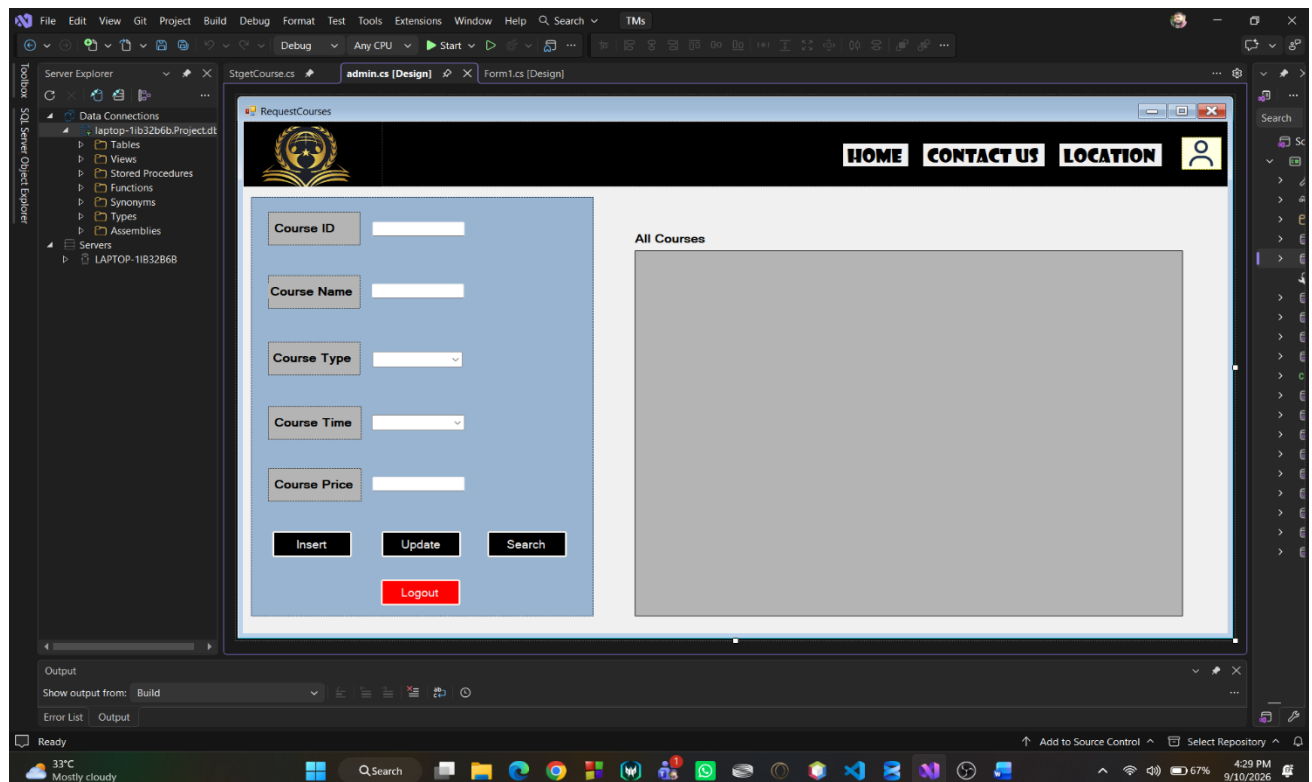
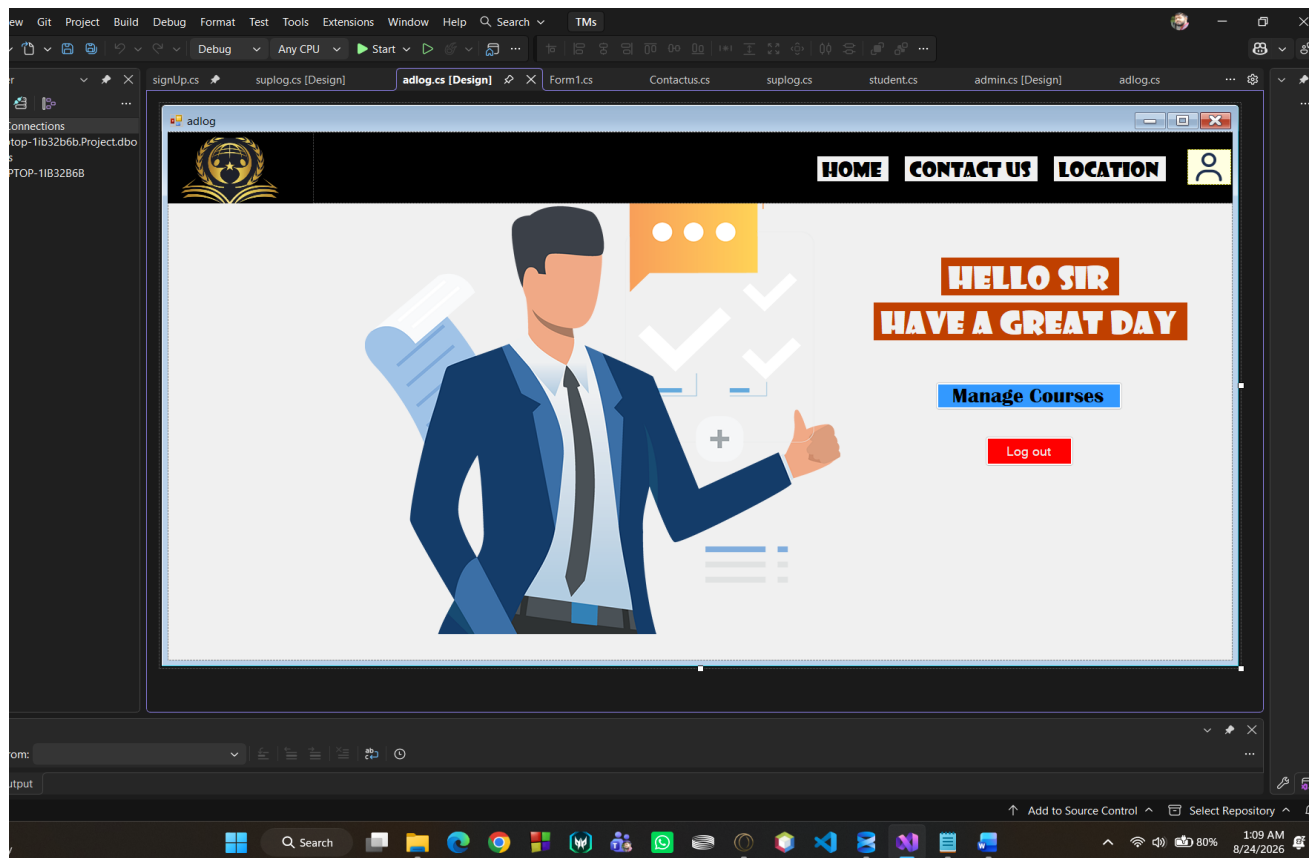
Chapter 3 — UI Navigation Diagram: (Demo) working on progress

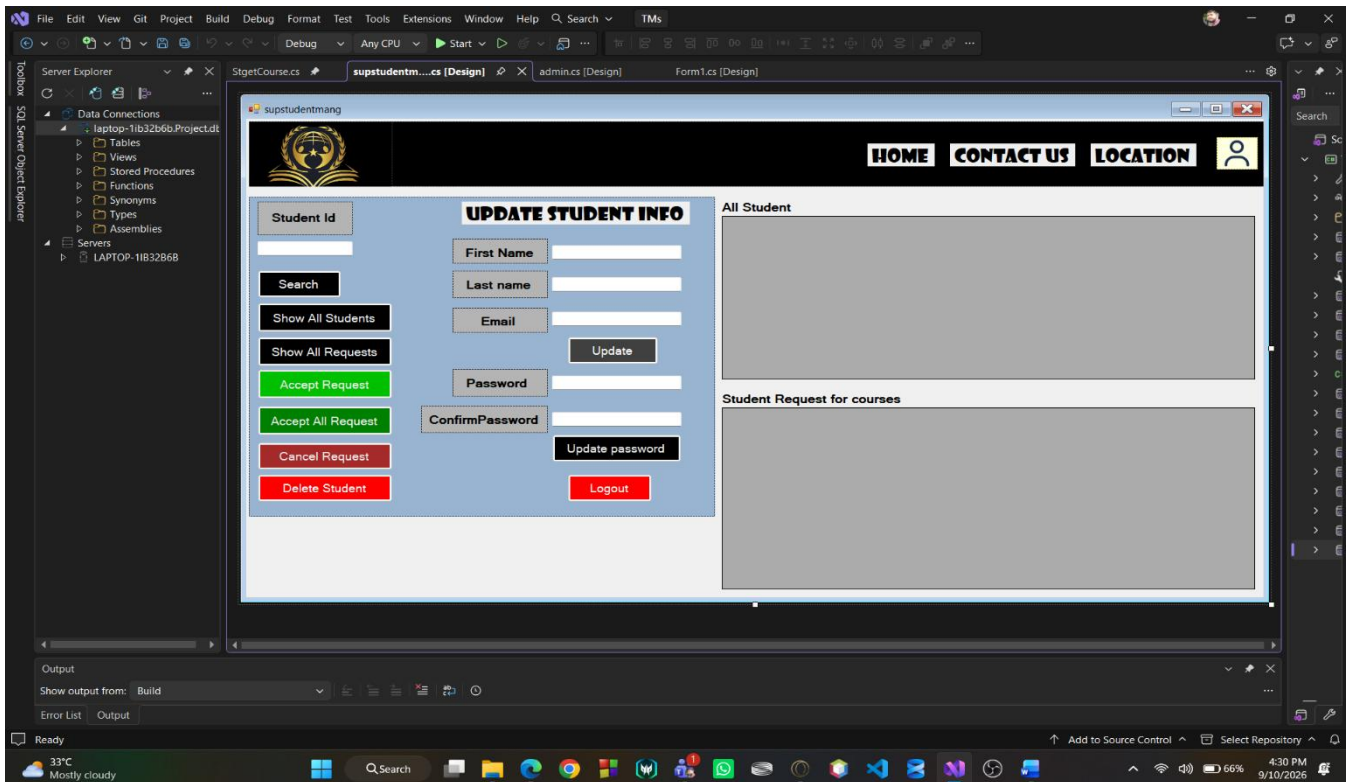
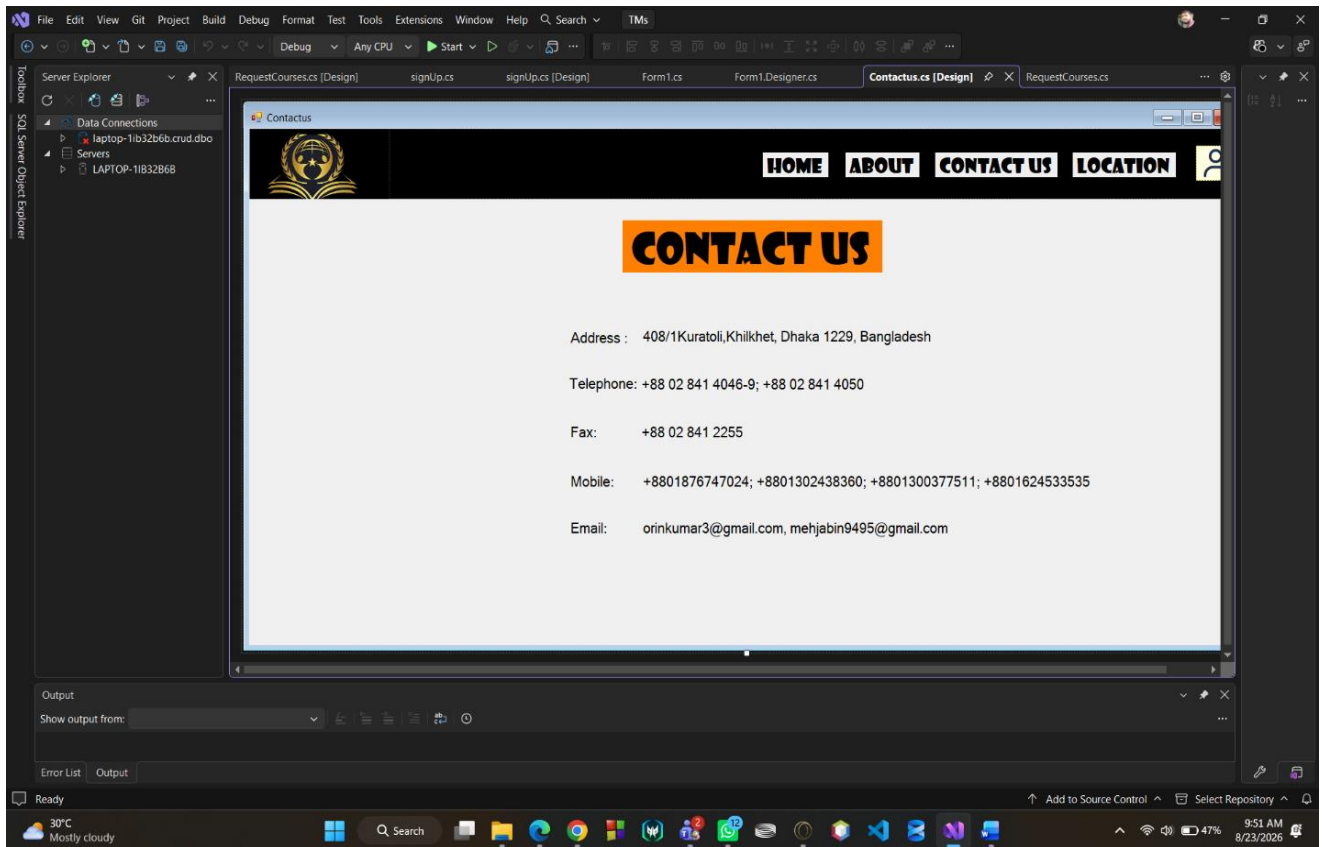


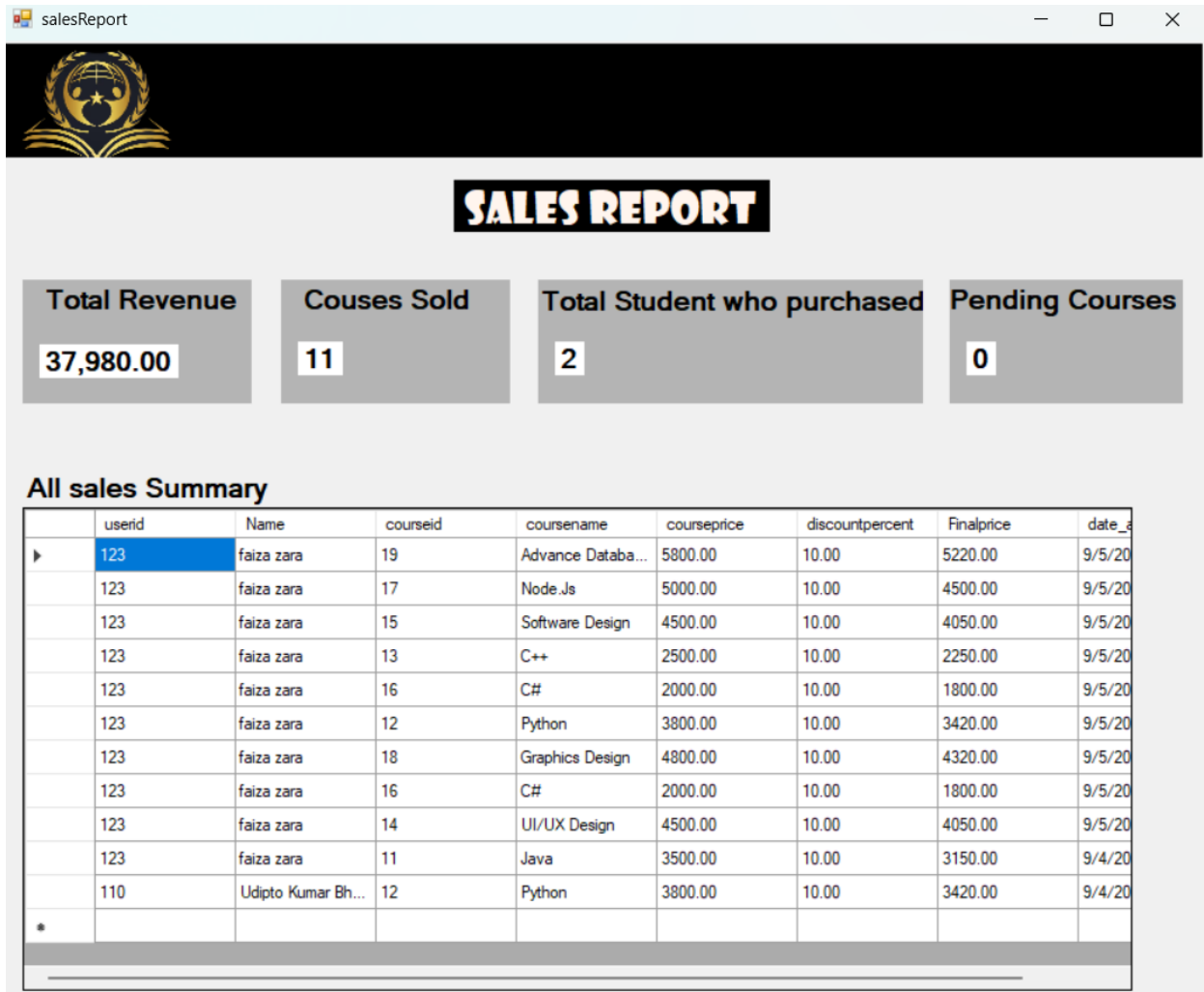












Chapter 4 — Database Design

4.1 Proposed Database

The system will use a relational database such as Microsoft SQL Server. The database will be designed to reduce duplication and maintain relationships between users, providers, courses, carts, orders, reviews, and offers.

4.2 Main Tables

Users

Column	Data Type	Constraint
--------	-----------	------------

UserId	INT	Primary Key, Identity
FullName	VARCHAR(100)	NOT NULL
Email	VARCHAR(100)	UNIQUE, NOT NULL
Password	VARCHAR(100)	NOT NULL
Phone	VARCHAR(20)	NOT NULL
Address	VARCHAR(200)	NULL
UserType	VARCHAR(20)	NOT NULL
Status	VARCHAR(20)	NOT NULL
CreatedAt	DATETIME	NOT NULL

CourseProviders

Column	Data Type	Constraint
ProviderId	INT	Primary Key, Identity
UserId	INT	Foreign Key
ProviderName	VARCHAR(100)	NOT NULL
Contact	VARCHAR(30)	NOT NULL
Address	VARCHAR(200)	NULL
Status	VARCHAR(20)	NOT NULL

Categories

Column	Data Type	Constraint
CategoryId	INT	Primary Key, Identity
CategoryName	VARCHAR(100)	UNIQUE, NOT NULL

Courses

Column	Data Type	Constraint
CourseId	INT	Primary Key, Identity
ProviderId	INT	Foreign Key
CategoryId	INT	Foreign Key
CourseName	VARCHAR(150)	NOT NULL
CourseType	VARCHAR(50)	NOT NULL
Price	DECIMAL(10,2)	CHECK Price >= 0
AvailableSeats	INT	CHECK AvailableSeats >= 0
MinSeats	INT	CHECK MinSeats >= 0
CourseTime	VARCHAR(100)	NULL
Description	VARCHAR(500)	NULL

Cart

Column	Data Type	Constraint
CartId	INT	Primary Key, Identity
CustomerId	INT	Foreign Key
CourseId	INT	Foreign Key
Quantity	INT	CHECK Quantity > 0
AddedDate	DATETIME	NOT NULL

Orders

Column	Data Type	Constraint
OrderId	INT	Primary Key, Identity
CustomerId	INT	Foreign Key
OrderDate	DATETIME	NOT NULL
TotalAmount	DECIMAL(10,2)	NOT NULL

PaymentMethod VARCHAR(50) NOT NULL

Status VARCHAR(30) NOT NULL

OrderItems

Column	Data Type	Constraint
OrderItemId	INT	Primary Key, Identity
OrderId	INT	Foreign Key
CourseId	INT	Foreign Key
Quantity	INT	CHECK Quantity > 0
UnitPrice	DECIMAL(10,2)	NOT NULL
Subtotal	DECIMAL(10,2)	NOT NULL

Reviews

Column	Data Type	Constraint
ReviewId	INT	Primary Key, Identity
CustomerId	INT	Foreign Key
CourseId	INT	Foreign Key
Rating	INT	CHECK Rating BETWEEN 1 AND 5
Comment	VARCHAR(500)	NULL
ReviewDate	DATETIME	NOT NULL

Offers

Column	Data Type	Constraint
OfferId	INT	Primary Key, Identity
CourseId	INT	Foreign Key

DiscountPercent DECIMAL(5,2) CHECK 0–100

StartDate DATE NOT NULL

EndDate DATE NOT NULL

4.3 Relationships

One Course Provider can have many Courses.

One Category can contain many Courses.

One Student can have multiple Cart records.

One Student can create multiple Orders.

One Order can contain multiple OrderItems.

One Course can appear in multiple OrderItems.

One Student can write multiple Reviews.

One Course can receive multiple Reviews.

One Course can have multiple Offers.

The OrderItems table is used to represent the many-to-many relationship between Orders and Courses.

4.4 Normalisation

The database will be designed according to Third Normal Form (3NF). Each table will represent a specific entity, and repeating groups will be avoided. Course information will not be duplicated inside Orders. Instead, OrderItems will store the individual courses belonging to each order. Foreign keys will connect related entities, while primary keys uniquely identify each record.

Chapter 5 — SQL Queries

```
CREATE DATABASE Project;
```

```
GO
```

```
USE Project;
```

```
GO
```

```
CREATE TABLE user_table
```

```
(
```

```
    userid    INT          PRIMARY KEY,  
    firstname VARCHAR(50)  NOT NULL,  
    lastname  VARCHAR(50)  NOT NULL,  
    email     VARCHAR(100) NOT NULL,  
    gender    VARCHAR(10)  NULL,  
    password  VARCHAR(50)  NOT NULL,  
    role      VARCHAR(20)  NOT NULL,  
    dob       DATE         NULL
```

```
);
```

```
GO
```

```
CREATE TABLE course_table
```

```
(
```

```
    courseid    INT          PRIMARY KEY,  
    coursename  VARCHAR(50)  NOT NULL,  
    coursetype  VARCHAR(50)  NOT NULL,  
    coursetime  VARCHAR(100) NOT NULL,  
    courseprice DECIMAL(10,2) NOT NULL DEFAULT 0,  
    discountpercent DECIMAL(5,2) NOT NULL DEFAULT 0
```

```
);
```

```
GO
```

```
CREATE TABLE cart_table
(
    cartid    INT IDENTITY(1,1) PRIMARY KEY,
    userid    INT          NOT NULL,
    courseid  INT          NOT NULL,
    status    VARCHAR(20)  NOT NULL DEFAULT 'Pending',
    date_added DATETIME    NOT NULL DEFAULT GETDATE(),

    CONSTRAINT FK_cart_user
        FOREIGN KEY (userid) REFERENCES user_table(userid),

    CONSTRAINT FK_cart_course
        FOREIGN KEY (courseid) REFERENCES course_table(courseid)
);
GO
```

```
INSERT INTO user_table
(userid, firstname, lastname, email, gender, password, role)
VALUES
(1, 'System', 'Admin', 'superadmin@gmail.com', 'Male', 'admin123', 'Super Admin');
```

```
INSERT INTO user_table
(userid, firstname, lastname, email, gender, password, role)
VALUES
(2, 'Course', 'Admin', 'admin@gmail.com', 'Male', 'admin456', 'Admin');
GO
```

```
INSERT INTO course_table (courseid, coursename, coursetype, coursetime, courseprice, discountpercent) VALUES
```

```
(11, 'Java', 'Offline Class', 'Sun 8:0 - Sun 10:20 AM', 3500.00, 10.00),
```

```
(12, 'Python', 'Online Class', 'Mon 11:20 - Mon 12:50 PM', 3800.00, 10.00),
```

```
(13, 'C++', 'Offline Class', 'Sun 11:20 - Sun 12:50 PM', 2500.00, 10.00),
```

```
(14, 'UI/UX Design', 'Offline Class', 'Sun 11:20 - Sun 12:50 PM', 4500.00, 10.00),
```

```
(15, 'Software Design', 'Online Class', 'Mon 11:20 - Mon 12:50 PM', 4500.00, 10.00),
```

```
(16, 'C#', 'Online Class', 'Mon 8:0 - Mon 9:30 AM', 2500.00, 10.00),
```

```
(17, 'Node.js', 'Offline Class', 'Mon 8:0 - Mon 9:30 AM', 5000.00, 10.00),
```

```
(18, 'Graphics Design', 'Online Class', 'Tue 8:0 - Tue 10:0 AM', 4800.00, 10.00);
```

```
GO
```

```
SELECT * FROM user_table;
```

```
SELECT * FROM course_table;
```

```
SELECT * FROM cart_table;
```

```
SELECT userid, password, role
```

```
FROM user_table
```

```
WHERE userid = @userid AND password = @password;
```

```
INSERT INTO user_table (userid, firstname, lastname, email, password, role, dob)
```

```
VALUES (@userid, @firstname, @lastname, @email, @password, @role, @dob);
```

```
SELECT * FROM course_table;
```

```
SELECT * FROM course_table WHERE courseid = @courseid;
```

```
SELECT COUNT(*) FROM course_table WHERE courseid = @courseid;
```

```
INSERT INTO course_table (courseid, coursename, coursetype, coursetime, courseprice)  
VALUES (@courseid, @coursename, @coursetype, @coursetime, @courseprice);
```

```
UPDATE course_table  
SET coursename = @coursename,  
    coursetype = @coursetype,  
    coursetime = @coursetime,  
    courseprice = @courseprice  
WHERE courseid = @courseid;
```

```
SELECT courseid, coursename, coursetype, coursetime, courseprice, discountpercent,  
    CAST(courseprice - (courseprice * discountpercent / 100) AS DECIMAL(10,2)) AS  
Finalprice  
FROM course_table;
```

```
SELECT * FROM course_table WHERE courseid = @courseid;
```

```
SELECT COUNT(*) FROM course_table WHERE courseid = @courseid;
```

```
INSERT INTO course_table (courseid, coursename, coursetype, coursetime, courseprice)  
VALUES (@courseid, @coursename, @coursetype, @coursetime, @courseprice);
```

```
UPDATE course_table  
SET coursename = @coursename,  
    coursetype = @coursetype,
```

```
    coursetime = @coursetime,  
    courseprice = @courseprice  
WHERE courseid = @courseid;
```

```
UPDATE course_table  
SET discountpercent = @discount  
WHERE courseid = @courseid;
```

```
UPDATE course_table  
SET discountpercent = @discount;
```

```
DELETE FROM course_table WHERE courseid = @courseid;
```

```
SELECT userid, firstname, lastname, email, dob, password  
FROM user_table  
WHERE role = 'Student';
```

```
SELECT userid, firstname, lastname, email, dob, password  
FROM user_table  
WHERE role = 'Student' AND userid = @userid;
```

```
UPDATE user_table  
SET firstname = @firstname,  
    lastname = @lastname,  
    email = @email,  
    dob = @dob
```

```
WHERE userid = @userid;
```

```
UPDATE user_table
```

```
SET password = @newpassword
```

```
WHERE userid = @userid;
```

```
DELETE FROM user_table WHERE userid = @userid;
```

```
SELECT c.cartid, c.userid, u.firstname, u.lastname,
```

```
       c.courseid, co.coursename, co.courseprice,
```

```
       c.status, c.date_added
```

```
FROM cart_table c
```

```
JOIN user_table u  ON c.userid = u.userid
```

```
JOIN course_table co ON c.courseid = co.courseid
```

```
WHERE c.status = 'Pending'
```

```
ORDER BY c.date_added;
```

```
UPDATE cart_table SET status = 'Approved' WHERE cartid = @cartid;
```

```
UPDATE cart_table SET status = 'Approved' WHERE status = 'Pending';
```

```
UPDATE cart_table SET status = 'Rejected' WHERE cartid = @cartid;
```

```
SELECT ISNULL(SUM(co.courseprice - (co.courseprice * co.discountpercent / 100)), 0)
```

```
FROM cart_table c
```

```
JOIN course_table co ON c.courseid = co.courseid
```

```
WHERE c.status = 'Purchased';
```

```
SELECT COUNT(*) FROM cart_table WHERE status = 'Purchased';
```

```
SELECT COUNT(DISTINCT userid) FROM cart_table WHERE status = 'Purchased';
```

```
SELECT COUNT(*) FROM cart_table WHERE status = 'Pending';
```

```
SELECT u.firstname, u.lastname, co.coursename,  
       CAST(co.courseprice - (co.courseprice * co.discountpercent / 100) AS DECIMAL(10,2)) AS  
Finalprice,  
       c.date_added  
FROM cart_table c  
JOIN user_table u  ON c.userid = u.userid  
JOIN course_table co ON c.courseid = co.courseid  
WHERE c.status = 'Purchased'  
ORDER BY c.date_added DESC;
```

```
SELECT co.courseid,  
       co.coursename,  
       COUNT(*) AS TimesPurchased,  
       SUM(co.courseprice - (co.courseprice * co.discountpercent / 100)) AS TotalRevenue  
FROM cart_table c  
JOIN course_table co ON c.courseid = co.courseid  
WHERE c.status = 'Purchased'  
GROUP BY co.courseid, co.coursename  
ORDER BY TotalRevenue DESC;
```

```
SELECT courseid, coursename, coursetype, coursetime, courseprice, discountpercent,
```

```
CAST(courseprice - (courseprice * discountpercent / 100) AS DECIMAL(10,2)) AS  
Finalprice
```

```
FROM course_table;
```

```
SELECT * FROM course_table WHERE courseid = @courseid;
```

```
SELECT COUNT(*) FROM cart_table
```

```
WHERE userid = @userid AND courseid = @courseid
```

```
AND status IN ('Pending', 'Approved', 'Purchased');
```

```
INSERT INTO cart_table (userid, courseid, status, date_added)
```

```
VALUES (@userid, @courseid, 'Pending', GETDATE());
```

```
SELECT ROW_NUMBER() OVER (PARTITION BY c.userid ORDER BY c.cartid) AS  
DisplayNo,
```

```
c.cartid, c.courseid, co.coursename, co.courseprice, co.discountpercent,
```

```
CAST(co.courseprice - (co.courseprice * co.discountpercent / 100) AS DECIMAL(10,2)) AS  
Finalprice,
```

```
c.status, c.date_added
```

```
FROM cart_table c
```

```
JOIN course_table co ON c.courseid = co.courseid
```

```
WHERE c.userid = @userid;
```

```
DELETE FROM cart_table
```

```
WHERE cartid = @cartid AND userid = @userid;
```

```
UPDATE cart_table SET status = 'Purchased'
```

```
WHERE cartid = @cartid AND userid = @userid AND status = 'Approved';
```

```
UPDATE cart_table SET status = 'Purchased'
```

```
WHERE userid = @userid AND status = 'Approved';
```

```
SELECT userid, firstname, lastname, email, password, dob
```

```
FROM user_table
```

```
WHERE userid = @userid;
```

```
UPDATE user_table
```

```
SET firstname = @firstname,
```

```
    lastname = @lastname,
```

```
    email = @email,
```

```
    dob = @dob
```

```
WHERE userid = @userid;
```

```
UPDATE user_table
```

```
SET password = @newpassword
```

```
WHERE userid = @userid;
```

```
SELECT u.userid, u.firstname, u.lastname, u.email,
```

```
       co.courseid, co.coursename, co.courseprice, co.discountpercent,
```

```
       CAST(co.courseprice - (co.courseprice * co.discountpercent / 100) AS DECIMAL(10,2)) AS  
discountprice,
```

```
       c.status, c.date_added
```

```
FROM cart_table c
```

```
JOIN user_table u ON c.userid = u.userid
```

```
JOIN course_table co ON c.courseid = co.courseid  
WHERE c.userid = @userid AND c.status = 'Purchased'  
ORDER BY c.date_added DESC;
```

```
SELECT u.userid, u.firstname, u.lastname, u.email,  
       co.courseid, co.coursename, co.courseprice, co.discountpercent,  
       CAST(co.courseprice - (co.courseprice * co.discountpercent / 100) AS DECIMAL(10,2)) AS  
discountprice,  
       c.status, c.date_added  
FROM cart_table c  
JOIN user_table u  ON c.userid = u.userid  
JOIN course_table co ON c.courseid = co.courseid  
WHERE c.cartid IN (@id0, @id1
```

```
SELECT c.cartid, c.userid, u.firstname, u.lastname, u.email,  
       c.courseid, co.coursename, co.courseprice, co.discountpercent,  
       CAST(co.courseprice - (co.courseprice * co.discountpercent / 100) AS DECIMAL(10,2)) AS  
Finalprice,  
       c.status, c.date_added  
FROM cart_table c  
JOIN user_table u  ON c.userid = u.userid  
JOIN course_table co ON c.courseid = co.courseid  
WHERE c.cartid IN (@id0, @id1, @id2
```

Chapter 8 — Conclusion & Future Work

The Course Management System is designed to provide a centralized platform for students and course providers. The proposed system combines course browsing, searching, filtering, cart management, enrollment, invoice generation, reviews, course management, provider management, sales reporting, and platform administration in one desktop application.

The system will use C# Windows Forms for the graphical user interface and SQL Server for database management. The three-role structure ensures that Super Admins, Course Providers, and Students have different permissions and responsibilities. Data isolation will also be maintained so that a Course Provider can access only their own courses, orders, reviews, and earnings.

Future development will focus on implementing the complete database connection, authentication and role-based routing, CRUD operations, cart and checkout functionality, invoice generation, review management, reporting, and validation. Additional features may include online payment integration, course video/material management, notification systems, attendance tracking, certificates, and more advanced reporting.

The final objective of the project is to create a user-friendly and organised course marketplace that reduces the difficulties students and course providers face when managing courses through separate or manual methods.